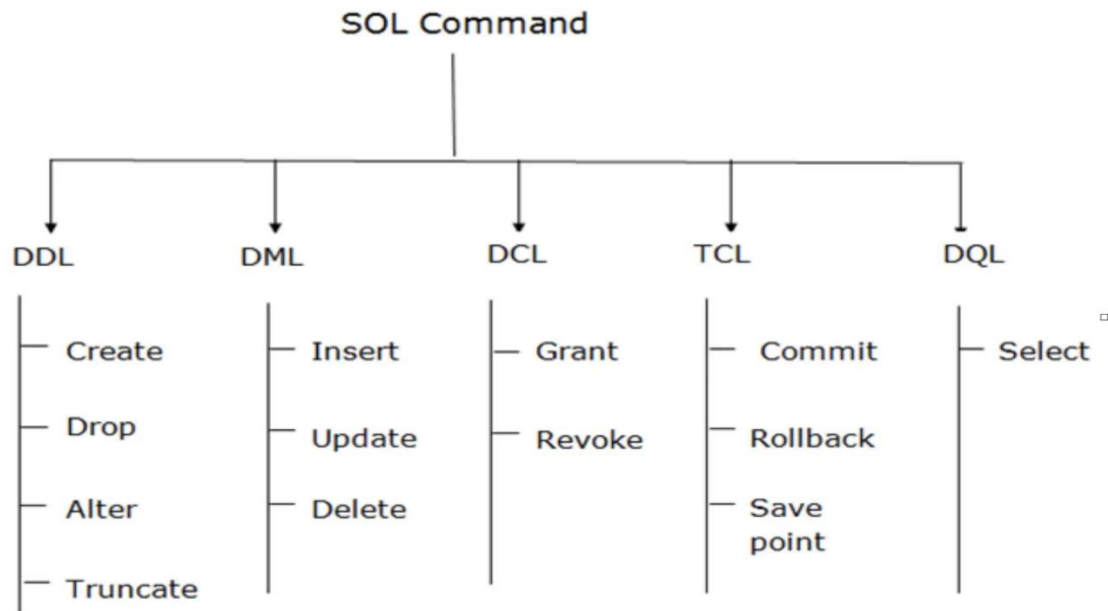


Classification of SQL commands

- commands can be used not only for searching the database but also to perform various other functions like, for example, you can create tables, add data to tables, or modify data, drop the table, set permissions for users.



DDL: Data Definition Language

DDL commands are auto-committed. That means it saves all the changes permanently in the database.

- DDL statements are used to build and modify the objects and structure of tables in database.
- The DDL part of SQL permits database tables to be created or deleted.
- It also defines indexes (keys), specifies links between tables, and imposes constraints between tables.

These SQL commands are used for creating, modifying, and dropping the structure of database objects. The commands are CREATE, ALTER, DROP, RENAME, and TRUNCATE

Command	Description
Create	Creates a new table, a view of a table, or other object in

	database
Alter	Alters the structure or Modifies an existing database object, such as a table.
truncate	remove all records from a table, including all spaces allocated for the records are removed
Drop	Deletes an entire table, a view of a table or other object in the database.
Rename	Rename a table, a view of a table or other object in the database.

◆ **DML: Data Manipulation Language**

DML statements allow changing data within the database. DML commands are not auto-committed. It means changes are not permanent to database, they can be rolled back. These SQL commands are used for storing, retrieving, modifying, and deleting data. These commands are SELECT, INSERT, UPDATE, and DELETE.

Command	Description
Insert	insert data (new row) into a table
Update	updates existing data(row) within a table
Delete	deletes all records from a table, the space for the records remain

◆ **TCL: Transaction Control Language**

TCL statements are used to manage the changes made by DML statements. It allows statements to be grouped together into logical transactions. These commands can cancel changes made by other commands by rolling back to original state. It can also make changes permanent.

Command	Description
Commit	to permanently save
rollback	restore database to original since the last COMMIT(to undo change)

Savepoint	identify a point in a transaction to which you can later roll back (to save temporarily)
	x

◆ DCL : Data Control Language

DCL statements control access to data and to the database. It is used to give the permission to user to use the database or object. It can also take back the permission. It allows central administration of user privileges and access. Data control language provides command to grant and take back authority.

Command	Description
Grant	gives user's access privileges to database
Revoke	withdraw access privileges given with the GRANT command (take back permission)

◆ DQL: Data Query Language

DQL allows getting data from the database. This command is heart of SQL. When SELECT is fired against a table or tables, the result is compiled into a temporary table.

Command	Description
Select	retrieve records from one or more table

◆ DDL: Data

Definition

Language

CREATE

Statement

SQL CREATE TABLE statement is used to create table in a database. It defines each column of the table uniquely. Each column has a minimum of three attributes, a name, data type and size (i.e. column width).

Each table column definition is separated from the other by a comma. SQL

statement is terminated with a semi colon. Whenever user creates table, the table name must be a legal SQL name and it must not conflict with the name of any of your existing tables.

Rules for Creating Tables

1. A name can have maximum up to 30 characters
2. Alphabets from A-Z, a-z and numbers from 0-9 are allowed
3. A name should begin with an alphabet
4. The use of the special character like _ is allowed and also recommended. (Special characters like \$, # are allowed only in Oracle).
5. SQL reserved words not allowed. For example: create, select, and so on.

Syntax:

```
CREATE TABLE < TableName >
(< ColumnName1 >< DataType >< size >,
< ColumnName2 >< DataType >< size >.....,
< ColumnNameN >< DataType >< size >);
```

Example : CREATE TABLE EMP
 (ENO NUMBER (3),
 ENAME VARCHAR2 (15),
 EMPADD VARCHAR2 (20));

Creating A Table From**Previously Created Table****Syntax:**

```
CREATE TABLE < TableName1 > (< ColumnName1 >, <
ColumnName2 > .....< ColumnNameN >) AS SELECT
<ColumnName1>, <ColumnName2>.....< ColumnNameN
> FROM < TableName2 > WHERE <condition>;
```

If the TableName2 is having records then the target table – Table Name1 will be occupied with the same records.

Example

1. CREATE TABLE EMP1 AS SELECT * FROM EMP;
2. Change column name at table creation time. For example-CURBAL to BALANCE.
3. CREATE TABLE EMP2

```
(ENO,ENMAE,EADDRESS) AS SELECT  
ENO,ENMAE,EMPADD FROM EMP;
```

➤ **Create Table without Records**

To create a target table without records from the source table (i.e. create the structure only) then the AS SELECT statement have to use WHERE clause with condition that cannot be satisfied.(i.e. SELECT statement in the CREATE TABLE will not retrieve any rows from the source code. It will just retrieve the table structure. Thus, the target table will be created empty.)

Syntax: CREATE TABLE <new table name> AS SELECT * FROM
<old table name> WHERE 1=2;

```
CREATE TABLE EMP3 (ENO, ENMAE, EADDRESS) AS SELECT  
ENO, ENMAE, EMPADD FROM EMP WHERE ENO=1;
```

🌐 **Alter Table (Modifying The Structure Of Table)**

- The ALTER TABLE statement is used to add, modify or delete columns in an existing table. It is also used to rename a table.
- You can also use ALTER TABLE command to add and drop various constraints on an existing table.
- ALTER TABLE can
 - add or delete columns
 - add or remove constraints
 - create or destroy indexes
 - change data types of columns
 - Rename columns.

Restrictions on the ALTER table

- 1) User cannot change the column name
- 2) User cannot decrease the size of column if record in the column.

➤ **Adding New Columns**

- If you want to single add columns in table

Syntax: ALTER TABLE table_name ADD column_name column-definition;

Example: ALTER TABLE EMP ADD (CITY VARCHAR2 (20));

- If you want to add multiple columns in table

Syntax: ALTER TABLE table_name ADD (column1
column_definition, column2 column_definition,
.....Column_n column-
definition);

Example: ALTER TABLE EMP ADD (CITY VARCHAR2 (20), GENDER CHAR(6));

- **Dropping a Column:** remove column from the
table

Syntax: ALTER TABLE <TableName> DROP COLUMN
<ColumnName>;

Example: ALTER TABLE EMP DROP COLUMN CITY;

- **Modifying Existing Columns**

- If you want to modify an existing column in table

Syntax: ALTER TABLE table_name MODIFY (<ColumnName>
<NewDatatype> <Size>);

Example: ALTER TABLE EMP MODIFY (NAME VARCHAR2 (40)
);

- If you want to modify multiple columns in table

Syntax: ALTER TABLE table_name MODIFY (<ColumnName1>
<NewDatatype><Size>,
<ColumnName2> <NewDatatype><Size> ... <ColumnName
n><NewDatatype> <Size>); **Example:** ALTER TABLE EMP1
MODIFY (ENAME VARCHAR2(20), EADD VARCHAR2(20));

- **RENAME Existing Column:** Alter table rename column

Syntax: ALTER TABLE table_name RENAME COLUMN old_name col to
new_name col;

Example: ALTER TABLE EMP RENAME COLUMN EMPADD TO
EMPADDRESS;

- **Making Column Unused**

Syntax: ALTER TABLE <Table_Name> SET UNUSED COLUMN
<Col>;

Example: ALTER TABLE STUDENT SET UNUSED COLUMN
MARKS;

Even though the column is unused still it will occupy memory.

- **DROPPING UNUSED COLUMNS**

Syntax: ALTER TABLE <table_name> DROP UNUSED COLUMNS;

Example: ALTER TABLE STUDENT DROP UNUSED COLUMNS;

You cannot drop individual unused columns of a table.

➤ **Alter Table To Add / Remove Constraints**

- To add a primary key constraint after creation of a table

Syntax: ALTER TABLE <TableName> ADD PRIMARY KEY (column(s));

Example: ALTER TABLE EMP ADD

PRIMARY KEY (ENO); ALTER TABLE

EMP ADD PRIMARY KEY (ID,

NAME);

It will add a primary key constraint to a specified columns .If there is more than one column to specify then all the columns are separated by commas and will create composite primary key.

ALTER TABLE EMP MODIFY SALARY DECIMAL (18, 2)
NOT NULL;

- To add a foreign key constraint after creation of a table

Example: ALTER TABLE EMP ADD FOREIGN KEY

(DEPTNO) REFERENCES DEPT (DEPTNO);

- To remove primary key constraint

Syntax: ALTER TABLE <TableName> DROP PRIMARY KEY;

Example: ALTER TABLE EMP DROP PRIMARY KEY;

It will remove primary key constraint from a table.

- To add user defined constraint

Example: ALTER TABLE EMP ADD CONSTRAINT

myUniqConstraint UNIQUE (AGE, SALARY);

It will add the constraint on the table.

- To remove user defined constraint

Syntax: ALTER TABLE <TableName> DROP CONSTRAINTS
<Const_Name>

Example: ALTER TABLE EMP DROP CONSTRAINT

myUniqConstraint; It will remove the constraint,

which was defined on the table.

⊕ **DROP TABLE (to drop the table from the database)**

DROP TABLE statement is used to remove a table definition

and all data, indexes, triggers, constraints, and permission specifications for that table. If a table is dropped, all records within table are lost and cannot be recovered. DROP TABLE statement once executed is permanent. So we have to use very careful this command. **Syntax:**

```
DROP TABLE < TableName >; OR DROP DATABASE  
DatabaseName;
```

Example: DROP TABLE EMP;

Note:

1. DROP statement removes the table.
2. It deletes all its data.
3. It deletes constraints and indexes associated with it.
4. Any views and synonyms are remain but are invalid.
5. Any pending Transaction is committed.
6. Only the creator of the table as user with the DROP ANY

TABLE privilege can remove a table.

⊕ **TRUNCATE Statement**

It is used to delete the entire table data permanently and free the containing space. Records cannot be ROLLBACK.

Syntax: TRUNCATE TABLE <Table_Name>;

Example: TRUNCATE TABLE EMP;

➤ **Difference b/w DROP and TRUNCATE statements**

When you drop a table:

- Table structure will be dropped
- Relationship will be dropped
- Integrity constraints will be dropped
- Access privileges will also be dropped

When we TRUNCATE a table, the table structure remains the same.

- Truncate operations drop recreate the table which much faster than deleting rows one by one.
- Truncate operations are not transaction safe.
- The numbers of deleted rows are not returned.

⊕ **RENAME Statement**

It is used to change the name of a table.

Syntax: RENAME < OldTableName > TO < NewTableName>;

OR

ALTER TABLE table_name RENAME TO new_table_name;

Example: RENAME EMP TO

EMPLOYEE;

OR ALTER TABLE EMP RENAME TO EMPLOYEE;

⊕ DESC (Viewing Table Structure)

It used to view structure of a table.

Syntax: DESCRIBE <Table_Name> OR DESC <Table_Name >**Example:** DESC EMP;**➤ Difference between DROP, DELETE and TRUNCATE**

Drop	Delete	Truncate
1. Whole structure and data will drop. 2.can't Rollback 3.can't use where clause 4.All data and structure drop simultaneously	1. Structure will not drop, data will drop. 2.can Rollback 3.can use WHERE clause 4.Row by Row data Delete	1. Structure will not drop, all data will drop. 2.can't Rollback 3.can't use WHERE clause 4. truncate operation drop and recreate the table.

◆ DML: Data**Manipulation****Language ⊕****INSERT****STATEMENT**

It is used to insert a single or a multiple records in a table. When inserting a single row of data into the table, the insert operation:

- Creates a new row (empty) in the database table
- Loads the values passed (by the SQL insert) into the specified columns

In INSERT INTO sentence, table columns and values have a one to one relationship.

i.e. First value is inserted into first column, second value is inserted into

second column and so on.

1. There are exactly same number of values as columns and
2. The values are in sequence exactly in accordance with data type of table columns.

Syntax: INSERT INTO <Table_Name> < ColumnName1 >, < ColumnName2 >.....< ColumnNameN > VALUES (< Value1 >, < Value2 >.....< Value N >);

Example: INSERT INTO EMP (ENO, ENAME, EADD) VALUES (1, 'ABHI', 'PUNE');

You can specify or ignore the column names while using INSERT INTO statement.

We have two methods to insert.

Ø By value method:

1. By specifying column names
2. Without specifying column names

Ø By address method

➤ **Using Value Method:**

- Inserting Data Into Specified Columns Using Value Method

Syntax: INSERT INTO <table_name> (col1, col2...Col n) VALUES (value1, value2... Value n);

Example: INSERT INTO STUDENT (ENO, ENAME) VALUES (5, 'RAMESH');

- Inserting Data Into Without specifying column names Using Value Method

Syntax: INSERT INTO TABLE_NAME VALUES (value1, value2, value 3, Value N);

Example: INSERT INTO EMP VALUES (1, 'SUDHA', 'SANGOLA'); INSERT INTO EMP VALUES (2, 'SAKETH', 'SOLAPUR');

To insert a new record again you have to type entire insert command, if there is lot of records this will be difficult. This will be avoided by using address method.

➤ **Using Address Method**

Syntax: INSERT INTO <TABLE_NAME> VALUES (&COL1,&COL2,&COL3....&COLN);

This will prompt you for the values but for every insert you have to use forward slash.

Example: INSERT INTO EMP VALUES (&ENO,
'&ENAME', &EADD); ENTER VALUE
FOR ENO: 3
ENTER VALUE FOR
ENAME: JAGAN
ENTER VALUE FOR
EADD: PUNE
OLD 1: INSERT INTO EMP VALUES (&ENO,
'&ENAME', &EADD) NEW 1: INSERT INTO
EMP VALUES (3, 'JAGAN', 'PUNE') SQL> /
ENTER VALUE FOR
ENO: 4 ENTER
VALUE FOR ENAME:
RAM ENTER VALUE
FOR EADD: MUMBAI
OLD 1: INSERT INTO EMP VALUES (&ENO,
'&ENAME', &EADD) NEW 1: INSERT INTO
EMP VALUES (4, 'RAM', 'MUMBAI')

➤ **Inserting Data Into A Table From Another Table**

Syntax: INSERT INTO tablename SELECT col_name, col_name
FROM table_name [WHERE <condition>];

Example: INSERT INTO EMP2 SELECT * FROM EMP1;

🌐 **SELECT (Retrieving Data Using Query)**

SQL SELECT Statement is used to fetch the data from a database table which returns data in the form of result table. These result tables are called result-sets. The SELECT command is used to retrieve rows selected from one or more tables.

➤ **All Rows and All Columns**

Syntax: SELECT * FROM <
 TableName >;
SELECT * FROM
EMP;

It will display all the data from all rows and columns from the table.

➤ **Filtering Table Data**

While viewing data from a table, it is rare that all the data from the table will be required each time. So, SQL provides a method of filtering table data that is not required. Ways for filtering data are:

1. Selected columns and all rows
2. Selected rows and all columns
3. Selected columns and selected rows

1. Selected Columns and All Rows

It will display all records with specific fields that user has specified with SELECT command.

Syntax: SELECT <Col_Name1>,<Col_Name2>,...<Col_NameN> FROM <
 TableName >;

Example: SELECT * FROM EMP;

2. Selected Rows and All Columns

Syntax: SELECT * FROM < TableName > WHERE < Condition >;

Example: SELECT * FROM EMP WHERE ENO=10;

The Oracle engine displays only those records that satisfy the specified condition after WHERE will become true. WHERE clause applies a filter on the rows retrieved.

When a WHERE clause is added to the SQL query, the Oracle engine compares each record in the table with the condition specified in WHERE clause. All operators such as logical, arithmetic, etc. can be used in the condition of WHERE clause.

3. Selected Columns and Selected Rows

Syntax: SELECT <Col_Name1>, <Col_Name2>,...<Col_Name N> FROM
 <TableName> WHERE < Condition > ;

Example: SELECT ENO, ENAME FROM EMP WHERE ENO=10;

It will display all records with specific fields after specified condition WHERE will be true.

➤ **ELIMINATING DUPLICATE ROWS**

DISTINCT command is used to retrieve only distinct or unique data. In a table, there may be a chance to exist a duplicate value and we want to retrieve only unique values. *SQL SELECT UNIQUE and SQL SELECT DISTINCT statements are same.*

Syntax: SELECT DISTINCT <Col_Name1>,
<Col_Name2>,...<Col_NameN> FROM <TableName>; **OR**
SELECT DISTINCT * FROM <TableName>;

Example: SELECT DISTINCT ENO, ENAME FROM EMP; **OR**
SELECT DISTINCT * FROM EMP;
DISTINCT clause allows removing duplicates from the result set. This clause can be used only with SELECT statement. It scans all the values of the specified columns and display only unique values. It eliminates rows that have exactly same contents in each column.

> Arithmetic calculation

Example: Select employee name, salary and compute salary * 0.05 for each row retrieved: SELECT ename, sal, sal * 0.05
"Salary" from emp;

Here the default output column sal*0.05 is renamed with Salary.

✚ DELETE (Deleting Data)

- The DELETE statement only deletes the rows from the table based on the condition defined by WHERE clause or delete all the rows from the table when condition is not specified. But it does not free the space containing by the table.
- But The TRUNCATE statement: it is used to delete all the rows from the table and free the containing space.

Syntax: DELETE FROM < Table_Name > [WHERE < condition >];

- If WHERE clause is not specified then all rows will be deleted.
- If WHERE clause is specified then it will delete all rows that satisfy the condition specified after WHERE.

Example: DELETE FROM EMP; // it will delete all the records from a CUST table.
DELETE FROM EMP WHERE CUSTNO > 10;

It will delete all the records for which CUSTNO is greater than 10.

✦ UPDATE (Updating A Contents Of A Table)

The UPDATE statement changes data in existing rows either by adding new data or modifying existing data.

User can update

- all rows from table
- selected rows from table

Syntax: UPDATE <Table_Name> SET
<ColumnName1>=<Expression1>,<ColumnName2>=
<Expression2>....., <ColumnNameN>=<ExpressionN> [WHERE
<Condition>] ;

OR

UPDATE table_name SET column_name = expression WHERE conditions

Example: UPDATE EMP SET CITY = 'BARODA';

- **All rows from table:** If you are not specifying any condition, this will update entire table. UPDATE EMP SET SALARY = 500;

UPDATE EMP SET ADDRESS = 'Pune', SALARY = 1000.00;

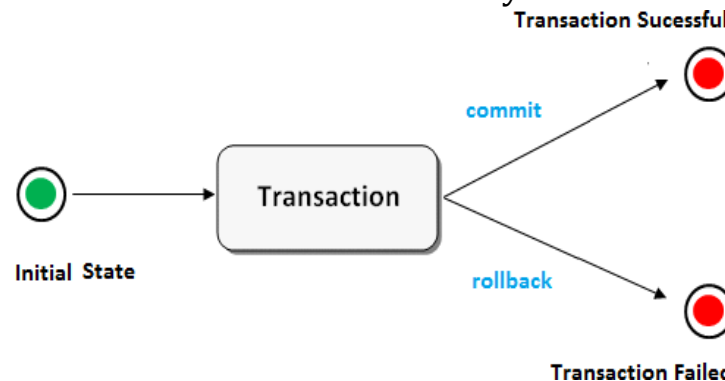
- **selected rows from table**

UPDATE EMP SET CITY = 'BARODA' WHERE ENO = 10;

✦ TCL: Transaction Control Language

TCL commands are used to manage transactions in database.

These are used to manage the change which is made in the database by a DML statement will be called as a transaction. We need to control these transactions made by DML statements. It also



allows statements to be grouped together into logical transactions.

Change will not reside permanently in the database, unless auto commit option enabled in the session. Those changes will reside in temporary memory. So, we need to handle these changes. TCL statements will be used for this purpose.

TCL statements are:

- COMMIT
- ROLLBACK
- SAVEPOINT

➤ **COMMIT (used to permanently save any transaction into database):**

The COMMIT command is used to save changes invoked by a transaction to the database. The COMMIT command saves all transactions to the database since the last COMMIT or ROLLBACK command. TCL are only used with the DML commands INSERT, UPDATE and DELETE only. They cannot be used while creating tables or dropping them because these operations are automatically committed in the database. **Syntax:** COMMIT;

If Auto commit option for session is enabled, then this is not required as commit takes place automatically. DDL statements will run implicit commit.

➤ **Rollback command (restores the database to last committed state)**

Rollback will be used to reject the changes made by DML statements in the database. If auto commit option enabled in the session, then there is no use with Rollback statements as transactions will be committed automatically.

If we give Rollback then any pending changes in the database which are made by DML statements will be rejected. It is also use with savepoint command to jump to a savepoint in a transaction.

Syntax: Rollback; OR rollback to savepoint-name;

➤ **Savepoint command**

Savepoint command is used to temporarily save a transaction so that you can rollback to that point whenever necessary.

Syntax: savepoint savepoint-name;

Example: *Savepoint and Rollback on class table,*

I D	NAM E
1	abhi
2	adam
4	alex

SQL> INSERT into class values
(5,'Rahul'); SQL> Commit;
SQL> UPDATE class set
name='abhijit' where id='5'; SQL>
savepoint A;
SQL> INSERT into class values
(6,'Chris'); SQL> savepoint B;
SQL> INSERT into class values
(7,'Bravo'); SQL> savepoint C;
SQL> SELECT * from class;

I D	NAM E
1	abhi
2	adam
4	alex
5	abhijit
6	chris
7	bravo

Now rollback to

savepoint B

SQL> Rollback o B;

SQL> SELECT * from class;

The resultant table will look like

ID	NAM E
-----------	------------------

1	abhi
2	adam
4	alex
5	abhijit
6	chris

ow rollback to

savepoint A

SQL> Rollback

to A;

SQL> SELECT *

from class; The

result table will

look like

I D	NAM E
1	abhi
2	adam
4	alex
5	abhijit

◆ DCL : Data Control Language

DCL commands are used to enforce database security in a multiple user database environment. These statements will be used by DBA people to control the access of users to the database. Only Database Administrator's or owners of the database object can provide / remove privileges on a database object. DCL statements:

1. GRANT
2. REVOKE

1) GRANT: GRANT is a command used to provide access or privileges on the database objects to the users. We can give any type of permissions to the users on the database using grant command.

Syntax: GRANT privilege_name ON object_name TO
{user_name|PUBLIC|role_name} [WITH GRANT OPTION];

privilege_name	It is the access right or privilege granted to the user. Some of the access rights are ALL, EXECUTE, and SELECT.
object_name	It is the name of a database object like TABLE, VIEW, STORED PROCEDURE and SEQUENCE.
user_name	It is the name of the user to whom an access right is being granted.
PUBLIC	It is used to grant access rights to all users.
ROLES	They are a set of privileges grouped together.
WITH GRANT OPTION	It allows a user to grant access rights to other users.

Example: GRANT SELECT ON EMP TO BCA;

This command grants a SELECT permission on employee table to user1.

- if we want to give Select permission on all the database tables to a user

Syntax: GRANT SELECT ANY TABLE TO user_name;

Example: GRANT SELECT ANY TABLE TO BCA;

- if we want to give select permission on specific table then **Syntax:** GRANT SELECT ON tab_name TO user_name; **Example:** GRANT SELECT ON EMP TO BCA;

- If we want to give Create table permission then, **Syntax:** GRANT CREATE TABLE TO user_name; **Example:** GRANT CREATE TABLE TO BCA;

Like this we can give any privilege to the users on the database.

2) REVOKE:

Revoke will be used to remove user access permissions which are granted previously to the users on the database.

Syntax: REVOKE privilege_name ON object_name FROM {user_name|PUBLIC|role_name};

Example: REVOKE SELECT ON EMP FROM BCA;

- if we want to give Select permission on all the database tables to a user,

Syntax: REVOKE SELECT ANY TABLE TO user_name;

Example: REVOKE SELECT ANY TABLE TO BCA;

- If we want to give select permission on specific table then **Syntax:** REVOKE SELECT ON tab_name TO user_name;

Example: REVOKE SELECT ON EMP TO BCA;

- If we want to give Create table permission then, **Syntax:** REVOKE CREATE TABLE TO user_name; **Example:** REVOKE CREATE TABLE TO BCA;

Like this we can give any privilege to the users on the database.

⊕ ROLES

- A role is a privilege (license) or set of privileges that allows a user to perform certain functions in the database.
- When there are many users in a database it becomes difficult to grant or revoke privileges to users.
- Therefore, if you define roles, you can grant or revoke privileges to users, thereby automatically granting or revoking privileges.

Syntax: CREATE ROLE role_name; **Example:** CREATE ROLE ASSISTANT;

- After creating role to assign permission to newly create role **Example:** GRANT CREATE TABLE, CREATE SESSION TO ASSISTANT;
- After assigning permission to role then you can that permission to user by using role. **Example:** GRANT ASSISTANT TO BCA;
- If WITH ADMIN OPTION is used, that user can then grant

roles to other users. **Syntax:** GRANT <ROLE_NAME> TO
<USER_NAME> WITH ADMIN OPTION; **Example:**
GRANT ASSISTANT TO BCA WITH ADMIN
OPTION;

- If role is removing then user cannot login because users have permission of the role.

Example: DROP ROLE ASSISTANT CASCADE;

Three type roles:

- Connect
- Resource
- DBA (or database administrator)

◆ Connect Role

The Connect role is the entry-level role. Connect role access can be granted permission with a database.

Example: GRANT CONNECT TO BCA; OR REVOKE CONNECT TO BCA;

◆ Resource Role

The Resource role gives the user more access to Oracle databases. In addition to the permissions that can be granted to the Connect role, Resource roles can also be granted permission to create procedures, triggers, and indexes.

Example: GRANT RESOURCE TO BCA; OR
REVOKE RESOURCE TO BCA;

◆ DBA Role

The DBA role includes all permission. Users with this role are able to do anything they want to the database system. You should keep the number of users with this role to a minimum to ensure system integrity.

Example: GRANT DBA TO BCA; OR REVOKE DBA TO BCA;

◆ **Creating Users:** Create a new user in the database.

Syntax:

```
CREATE USER user_name IDENTIFIED {BY password |
EXTERNALLY} [DEFAULT TABLESPACE tablespace_name]
[TEMPORARY TABLESPACE tablespace_name]
[QUOTA {integer [K|M] | UNLIMITED} ON tablespace] [PROFILE
profile];
```

Example: CREATE USER BCA

IDENTIFIED BY SANGOLA
DEFAULT TABLESPACE
SYSTEM
TEMPORARY
TABLESPACE
TEMP QUOTA
UNLIMITED ON
SYSTEM;

OR

CREATE USER BCA IDENTIFIED BY SANGOLA;

- To assigning permission to user one or more permission using without role command.

Syntax: GRANT <Permission> TO USERNAME;

Example: GRANT CONNECT TO BCA;

GRANT CONNECT, CREATE TABLE, CREATE SESSION
TO BCA;

- To change the user's password:

Example: ALTER USER BCA IDENTIFIED BY SANGOLA;

- To change the default table space:

Example: ALTER USER BCA DEFAULT TABLESPACE USERS;

- To remove a user entry from system database.

Syntax: DROP USER USER_NAME [CASCADE];

Example: DROP USER BCA CASCADE;

OR

DROP USER BCA;

If the CASCADE option is used, all objects owned by username are dropped along with the user's account. If CASCADE is not used then user still owns objects, that user is not dropped. This feature is confusing, but it is useful to drop users.

- To connect other user and another user table data and also perform DML operation.

CREATE USER BCA IDENTIFIED

BY SANGOLA; CREATE USER

BCS IDENTIFIED BY SANGOLA;

After creating two users then assign permission to both user.

```
CREATE ROLE ASSISTANT;  
GRANT CREATE TABLE, CREATE  
SESSION TO ASSISTANT; GRANT  
ASSISTANT TO BCA;
```

```
GRANT ASSISTANT TO BCS;
```

After granting permission then user bca create a

```
table stud CREATE TABLE STUD (ID INT,  
NAME VARCHAR2 (10));
```

Insert one record in the table stud

```
INSERT INTO STUD VALUES (1,'SHYAM');
```

Connect to another user **bcs** and use stud table for DML operation but before performing operation to grant DML permission to user **bcs**.

```
CONNECT BCA/SANGOLA
```

Syntax: GRANT <Permission> ON < table name>

```
TO <username>; GRANT SELECT, INSERT,  
UPDATE, DELETE ON STUD TO BCS;
```

```
CONNECT BCS/SANGOLA
```

```
INSERT INTO BCA.STUD  
VALUES (2,'RAM'); SELECT *  
FROM BCA.STUD;
```

When revoke permission then access is denied for

```
specified user CONNECT BCA/SANGOLA  
REVOKE SELECT ON STUD FROM BCS;
```

SQL operators and clauses

The symbols which are used to perform logical and mathematical operations in SQL are called SQL operators. SQL reserved words or characters are known as operators.

Generally there are three types of operators in SQL:

1. SQL Arithmetic Operators
2. SQL Comparison Operators
3. SQL Logical Operators

➤ **SQL Arithmetic Operators:**

Arithmetic operators in SQL are used to perform mathematical calculations like addition, subtraction, multiplication, division and modulus in SQL statements.

Arithmetic Operators/Operation	Example
+ (Addition)	A+B
- (Subtraction)	A-B
* (multiplication)	A*B
/ (Division)	A/B
% (Modulus)	A%B

➤ **SQL Comparison Operators:**

Relational operators in SQL are used to find the relation between two columns. i.e. to compare the values of two columns in SQL statements.

Operators	Example/Description
>	x > y (x is greater than y)
<	x < y (x is less than y)
>=	x >= y (x is greater than or equal to y)
<=	x <= y (x is less than or equal to y)
=	x == y (x is equal to y)
!= or < >	x != y or x <> y (x is not equal to y)
!<	x !< y (x is not less than y)
!>	x !> y (x is not greater than y)

➤ **SQL Logical Operators:**

Logical operators in SQL are used to perform logical operations on the given expressions in SQL statements. There are many operators in SQL which are used in SQL statements in the WHERE clause.

They are

Operator	Description
AND	The AND operator in SQL is used to compare data with more than one condition. If all the conditions returns TRUE then only it will display records.

OR	The OR operator in SQL is used to compare data with more than one condition. If either of condition is TRUE it will return data.
ALL	The ALL operator in SQL returns true when value matches with all values in single column set of values. It's like AND operator it will compare value against with all values in column.
ANY	The Any operator in SQL returns true when value matches with any value in single column set of values. It's like OR operator it will compare value against with any value in column.
LIKE	The LIKE operator in SQL is used to search for character string with specified pattern using wildcards in column.
IN	The IN operator in SQL is used to search for specified value matches any value in set of multiple values.
BETWEEN	The BETWEEN operator in SQL is used to get values within range.
EXISTS	The EXISTS operator in SQL is used to show result if subquery returns data.
NOT	The NOT operator in SQL is a negate operator that means it will show data for opposite of conditions what we mentioned in SQL statement.
SOME	The SOME operator in SQL is used to compare value with a single column set of values returned by subquery. SOME must match at least one value in subquery and that value must be preceded by comparison operators.

> **USING** =, >, <, >=, <=, !=, <>

Example: select * from student
 where no = 2; select *
 from student where no
 < 4; select * from


```
student where no > 3;
select * from student
where no <= 7; select *
from student where no
>= 3; select * from
student where no != 2;
select * from student
where no <> 2;
```

- **Using And:** This will give the output when all the conditions become true.

Syntax: select * from <table_name> where <condition1> and
<condition2> and....
<condition n>;

Example: select * from student where no = 2 **and** marks >= 200;

- **Using OR:** This will give the output when either one of the conditions becomes true.

Syntax: select * from <table_name> where <condition1> and
<condition2> or ..
<conditionn>;

Example: select * from student where no = 2 **or** marks >= 200;

- **NOT** will negate the condition. It will select the records
where condition is not satisfied.

- **Range Searching:** Oracle provides 3 predicates for searching data in a range.

1. BETWEEN
2. IN
3. IS

1) Using Between

- BETWEEN operator is use to select data that is within a range of values.
- BETWEEN operators select rows that contain values within specified lower and upper limit.
- Lower value must be written first. Two values in the range must be linked with keyword AND.
- BETWEEN operators can be used with numeric and character data

types.

Syntax: select * from <table_name> where <col> between <lower bound> and
<upper bound>;

Example: select * from student where marks between
200 and 400; SELECT EMPNO, ENAME, DEPTNO FROM
EMP WHERE DEPTNO>=15 AND DEPTNO<=25;

♦ Using Not Between

We can use NOT BETWEEN to select data which is not within the range.

Syntax: select * from <table_name> where <col> not between <lower bound> and
<upper bound>;

Example: select * from student where marks not between
200 and 400; SELECT * FROM EMP WHERE
EMPNO NOT BETWEEN 10 AND 20;

2) Using In

- If a value is required to be compared with list of values then IN predicate is used.
- IN predicate reduces the need to use multiple OR conditions.
- Arithmetic operator (=) compares a single value to another single value.

Syntax: select * from <table_name> where <col> in (value1, value2, value3...valuen);

Example: select * from student where no in (1, 2, 3);
SELECT EMPNO, ENAME, DEPTNO FROM EMP WHERE
DEPTNO 10 OR
DEPTNO=20;
SELECTEMPNO, ENAME, DEPTNO FROM EMPWHERE
DEPTNO IN (20,30);

- ♦ Using Not In: NOT IN predicate is will select all the rows where value does not match the list of values specified

Syntax: select * from <table_name> where <col> not in(value1, value2,value3...valuen);

Example: select * from student where no not in (1, 2, 3);
SELECT EMPNO, ENAME, DEPTNO FROM EMP WHERE
DEPTNO NOT IN (20,30);

- Using IS NULL: IS NULL condition is used to test for a NULL

value in the specified column.

Syntax: select * from <table_name> where <col> is null;

Example: select * from student where marks **is null**;

- 1) If expression is a NULL value, the condition evaluates to TRUE.
- 2) If expression is not a NULL value, the condition evaluates to FALSE.

➤ **Using Is Not Null:** IS NOT NULL can be used to test for NOT NULL value in the specified column.

Syntax: select * from <table_name> where <col> is not null;

Example: select * from student where marks **is not null**;

➤ **Using Like**

This will be used to search through the rows of database column based on the pattern you specify.

The LIKE keyword selects rows containing fields that match specified portions of character strings. LIKE is used with char, varchar, text, date time and small date time data. A wildcard allows the user to match fields that contain certain letters.

Syntax: select * from <table_name> where <col> like <pattern>;

%	Any string of zero or more characters
_	Any single character
[]	Any single character within the specified range (e.g., [a-f]) or set (e.g., [abcdef])
[^]	Any single character not within the specified range (e.g., [^a-f]) or set (e.g., [^abcdef])

Example:

- i) This will give the rows whose marks are 100. Select * from student where marks like 100;

NO	NAME	MARKS
1	Sudha	100

- ii) This will give the rows whose name

start with 'S'. Select * from
student where name like 'S%';

NO	NAME	MARKS
1	Sudha	100
2	Saketh	200

- iii) This will give the rows whose name
ends with 'h'. Select * from
student where name like '%h';

NO	NAME	MARKS
2	Saketh	200
3	Ramesh	100

- iv) This will give the rows whose name's second
letter start with 'a'. Select * from student
where name like '_a%';

NO	NAME	MARKS
2	Saketh	200
3	Ramesh	100
4	Jagan	300
5	Naren	400
6	Madhu	100
7	Rattu	300

- V) This will give the rows whose name's third letter
start with 'd'. Select * from student where
name like '__d%';

NO	NAME	MARKS
1	Sudha	100
6	Madhu	100

- Vi) This will give the rows whose name's second letter start with 't'
from ending. select * from student where name like '%_t%';

NO	NAME	MARKS
2	Saketh	200
7	Rattu	100

- Vii) This will give the rows whose name's third letter start with 'e' from

ending.

Select * from student where name like '%e_%';

NO	NAME	MARKS
2	Saketh	200
3	Ramesh	100

Viii) This will give the rows whose name

contains 2 a's. select * from student

where name like '%a% a %';

NO	NAME	MARKS
4	Jagan	300

You have to specify the patterns in like using underscore (_).

➤ **ORDER BY Clause**

The SQL ORDER BY clause is used for sorting data in ascending and descending order based on one or more columns. Some databases sort query results in ascending order by default.

Syntax:

SELECT expressions FROM tables WHERE conditions ORDER BY expression [ASC | DESC]; Sort the result in ascending order by NAME and SALARY.

SELECT * FROM EMP ORDER BY ENAME, SALARY;

To sort the result in descending order

by NAME. SELECT * FROM EMP

ORDER BY NAME DESC;

➤ **GROUP BY Clause**

- **GROUP BY** clause is another section of the **select** statement.
- This optional clause tells Oracle to group rows based on distinct values that exist for **specified columns**.
- GROUP BY clause creates a data set, containing several sets of records **grouped together** based on a condition.
- A query containing a group by clause is processed in the following way:
 1. Select all rows that satisfy the condition specified in the where clause.

2. From these rows form groups according to the group by clause.
3. Discard all groups that do not satisfy the condition in the having clause.
4. Apply aggregate functions to each group.
5. Retrieve values for the columns and aggregations listed in the select clause.

Syntax:

```
SELECT <ColumnName1>, <ColumnName2>, <ColumnNameN>,  
AGGREGATE_FUNCTION (<Expression>) FROM TableName WHERE  
<condition> GROUP BY <ColumnName1>,  
<ColumnName2>, <ColumnNameN>;
```

Example: SELECT DNO, SUM(SAL) FROM EMP GROUP BY DNO;

➤ **HAVING Clause**

- HAVING clause can be used in conjunction with the GROUP BY clause.
- HAVING imposes a condition on the GROUP BY clause, which further filters the groups created by the GROUP BY clause.
- Each column specification specified in the HAVING clause must occur within a statistical function or must occur in the list of columns named in the GROUP BY clause

Syntax:

```
SELECT <ColumnName1>, <ColumnName2>, <ColumnNameN>,  
AGGREGATE_FUNCTION (<Expression>) FROM TableName WHERE  
<condition> GROUP BY <ColumnName1>,  
<ColumnName2>, <ColumnNameN> HAVING  
AGGREGATE_FUNCTION (<Expression>)  
<operator> <value>;
```

Example: SELECT DNO, COUNT (ENO) FROM EMP GROUP
BY DNO HAVING COUNT (ENO) > 2;

Rules for Group By and having Clause

- Columns listed in the select statement have to be listed in the GROUP BY clause
- Columns listed in the GROUP BY clause need not be listed in the SELECT statement.

- Only group functions can be used in the HAVING clause.
- The group functions listed in the having clause need not be listed in the SELECT statement.

Where	Having
1. Without group by clause 2. Where clause select Row before grouping. 3. Where clause can't contain aggregate function.	1. With group by clause 2. Having clause select after grouping 3. Having clause contain aggregate Function.

SQL Functions

Oracle functions can be used to manipulate data and returning the result. A function is performs a specific task. A function can accept user-supplied variables or constants and perform operation on them. Such variables / constants are known as *arguments*.

Function_Name [(argument-1, argument-2,..)]

Functions are

- 1) Scalar Function (Single Row Function)
- 2) Aggregate (Group Function)
- 3) Miscellaneous Function

➤ **Single Row Functions**

- Functions that work on only one value at a time are called SCALAR Functions.
- A single row function returns one result for every row of table / view mentioned in query.

Example: LENGTH function calculates length of one string value.

- Single row functions can be grouped together depending on the data type of their arguments and return values.

Example: LENGTH function is related to string data type.

Functions can be classified according to different data types are:

1. Numeric Functions (For Number data type)
2. Character / String Functions (For String data type)
3. Date Function (For Date data type)
4. Conversion Functions (For Conversion of one data type to

another)

➤ **Aggregate functions**

- Functions that act on a set of values are called Aggregate / Group Functions. Example: SUM function calculates total of a column.
- Group function returns a single result row for a group of rows.
- Aggregate functions can appear in SELECT lists and in ORDER BY and HAVING clauses. They are commonly used with the GROUP BY clause in a SELECT statement.
- Aggregate functions perform a calculation on a set of values and return a single, or summary, value.

Function	Description
AVG	Returns the average value of the given column or expression Ex: SELECT AVG (salary) AS 'Average Salary' FROM emp;
COUNT	Number of rows where the value of the column is not NULL Ex: SELECT COUNT (dno) AS 'Number of Deptwise ' FROM emp;
COUNT(*)	Number of rows returned including duplicates and NULLs Ex: SELECT COUNT (*) FROM Employees WHERE eno = 5;
MAX	Return Maximum value of the given column or expression. MAX can be used with numeric, character and date time columns. With character columns, MAX finds the highest value in the collating sequence. MAX ignores any null values. Ex: SELECT MAX (salary) FROM Emp;
MIN	Return Minimum value of the given column or expression. MIN can be
	used with numeric, character and date time columns. With
	character columns, MIN finds the value that is lowest in the sort
	sequence. MIN ignores any null values. Ex: SELECT MIN (salary) FROM emp;
SUM	Returns the sum of all the values given column or expression. SUM can be used with numeric columns only. Ex: SELECT SUM (salary) AS 'Total Salary' FROM emp;

➤ **Numeric Functions (For Number data type)**

- **ABS:** Absolute value is the measure of the magnitude of value. Absolute value is always a positive number.

Syntax: `abs (value)`

Ex: `select abs(5), abs(-5), abs(0), abs(null) from dual;`
 ABS(5) ABS(-5) ABS(0) ABS(NULL)

 5 -5 0

- **SIGN:** Sign gives the sign of a value. Syntax:

`sign (value)`

Ex: `select sign(5), sign(-5), sign(0), sign(null) from dual;`
 SIGN(5) SIGN(-5)
 SIGN(0) SIGN(NULL)

 1 -1 0

- **SQRT:** This will give the square root of the given value. Syntax: `sqrt (value)` -- here value must be positive.

Ex: `select sqrt(4), sqrt(0), sqrt(null), sqrt(1) from dual;`
 SQRT(4) SQRT(0) SQRT(NULL) SQRT(1)

 2 0 1

- **MOD:** This will give the remainder. Syntax: `mod (value, divisor)`

Ex: `select mod(7,4), mod(1,5), mod(null,null), mod(0,0), mod(-7,4) from dual;`
 MOD(7,4) MOD(1,5) MOD(NULL,NULL) MOD(0,0) MOD(-7,4)

 3 1 0 -3

- **NVL:** This will substitute the specified value in the place of null values. Syntax: `nvl (null_col, replacement_value)`

Ex: `select * from student;` -- here for 3rd row marks value is null

 NO NAME MARKS
 1 a 100
 2 b 200

```

      3      c
select no, name, nvl(marks,300)
from student;

```

```

NO      NAME
      NVL(MARKS,300)

```

```

1      a      100
2      b      200
3      c      300

```

```
select nvl(1,2), nvl(2,3), nvl(4,3), nvl(5,4) from dual;
```

```

NVL(1,2)      NVL(2,3)      NVL(4,3)      NVL(5,4)
-----
1              2              4              5

```

```
select nvl(0,0), nvl(1,1), nvl(null,null), nvl(4,4) from dual;
```

```

NVL(0,0)      NVL(1,1)      NVL(null,null)      NVL(4,4)
-----
0              1              4

```

- **POWER:** Power is the ability to raise a value to a given exponent. Syntax: power (*value*, *exponent*)

Ex: select power(2,5), power(0,0), power(null,null), power(2,-5) from dual;

```

POWER(2,5)  POWER(0,0)      POWER(NULL,NULL)
POWER(2,-5)

```

```

32
1 .03125

```

- **EXP:** This will raise e value to the give power. Syntax: exp (*value*)

Ex: select exp(1), exp(2), exp(0), exp(null), exp(-2) from dual; EXP(1) EXP(2)

```

EXP(0)  EXP(NULL)      EXP(-2)

```

```

2.71828183  7.3890561      1 .135335283

```

- **CEIL:** This will produce a whole number that is greater than or equal to the specified value.

Syntax: ceil (*value*)

Ex: select ceil(5), ceil(5.1), ceil(-5), ceil(-5.1), ceil(0),

ceil(null) from dual; CEIL(5) CEIL(5.1) CEIL(-5)
CEIL(-5.1) CEIL(0) CEIL(NULL)

5 6 -5 -5 0

- **FLOOR:** This will produce a whole number that is less than or equal to the specified value.

Syntax: floor (*value*)

Ex: select floor(5), floor(5.1), floor(-5), floor(-5.1), floor(0),
floor(null) from dual;

FLOOR(5) FLOOR(5.1) FLOOR(-5) FLOOR(-5.1)
FLOOR(0) FLOOR(NULL)

5 5 -5 -6 0

- **ROUND:** This will rounds numbers to a given number of digits of precision. Syntax: round (*value, precision*)

Ex: select round(123.2345), round(123.2345,2),
round(123.2354,2) from dual; ROUND(123.2345)

ROUND(123.2345,0) ROUND(123.2345,2) ROUND(123.2345,-
2)

123 123 123.23 100

- **TRUNC:** This will truncates or chops off digits of precision from a number. Syntax: trunc (*value, precision*)

Ex: select trunc(123.2345), trunc(123.2345,2), trunc(123.2354,2)
from dual; TRUNC(123.2345) TRUNC(123.2345,2)

TRUNC(123.2345,-1) TRUNC(123.2345,-2)

123 123.23 120 100

- **GREATEST:** This will give the greatest number. Syntax: greatest (*value1, value2, value3 ... valuen*)

Ex: select greatest (1, 2, 3), greatest (-1, -2, -3) from dual;
GREATEST (1,2,3) GREATEST(-1,-2,-3)

- Ø If all the values are zeros then it will display zero.
- Ø If any of one or all the parameters are nulls then it will display nothing.

- number. Syntax: least (*value1*, *value2*, *value3* ... *valuen*)

- Ø If all the values are zeros then it will display zero.
- Ø If any of one or all the parameters are nulls then it will display nothing.

- **INITCAP:** This will capitalize the initial letter of the string. Syntax: `initcap (string)`

- **UPPER:** This will convert the string into uppercase. Syntax: upper (*string*)

- **LOWER:** This will convert the string into lowercase. Syntax: lower (*string*)

- **LENGTH:** This will give length of the string. Syntax: length (*string*)
Ex: select length('computer') from dual;
LENGTH

8

- **RPAD:** This will allows you to pad the right side of a column with any set of characters.

Syntax: rpad (*string, length [, padding_char]*)

Ex: select rpad('computer',15,'*'),
rpad('computer',15,'*#') from dual;
RPAD('COMPUTER' RPAD('COMPUTER'

computer***** computer*#*#*#*

-- Default padding character was blank space.

- **LPAD:** This will allows you to pad the left side of a column with any set of characters.

Syntax: lpad (*string, length [, padding_char]*)

Ex: select lpad('computer',15,'*'),
lpad('computer',15,'*#') from dual;
LPAD('COMPUTER' LPAD('COMPUTER'

*****computer *#*#*#*computer

-- Default padding character was blank space.

- **LTRIM:** This will trim off unwanted characters from the left end of string. Syntax: ltrim (*string [,unwanted_chars]*)

Ex: select ltrim('computer','co'),
ltrim('computer','com') from dual;
LTRIM(LTRIM

mputer puter

select ltrim('computer','puter'),
ltrim('computer','omputer') from dual;
LTRIM('Csel LTRIM('C

computer computer

-- If you haven't specify any unwanted characters it will display entire string.

- **RTRIM:** This will trim off unwanted characters from the right end of string. Syntax: rtrim (*string* [, *unwanted_chars*])

Ex: select rtrim('computer','er'),
rtrim('computer','ter') from dual;

RTRIM(RTRIM

```
-----
comput  compu
select rtrim('computer','comput'),
rtrim('computer','compute') from dual;
RTRIM('C  RTRIM('C
```

```
-----
computer      computer
```

-- If you haven't specify any unwanted characters it will display entire string.

- **TRIM:** This will trim off unwanted characters from the both sides of string. Syntax: trim (*unwanted_chars* from *string*)

Ex: select trim('i' from 'indiani') from dual;

TRIM(

```
-----
ndian
select trim( leading'i' from 'indiani') from dual;
```

-- this will work as LTRIM TRIM(L

```
-----
Ndiani
```

select trim(trailing'i' from 'indiani') from dual;

-- this will work as RTRIM TRIM(T

```
-----
Indian
```

- **TRANSLATE:** This will replace the set of characters, character by character. Syntax: translate (*string*, *old_chars*, *new_chars*)

Ex: select translate('india','in','xy') from dual;

TRANS

```
-----
```

xydxa

- **REPLACE:** This will replace the set of characters, string by string. Syntax: replace (*string*, *old_chars* [, *new_chars*])

Ex: select replace('india','in','xy'),
replace('india','in') from dual; REPLACE
REPLACE

Xydia dia

- **CONCAT:** This will be used to combine two strings only. Syntax: concat (*string1*, *string2*)

Ex: select concat('computer',' operator') from dual;
CONCAT('COMPUTER'
computer operator

If you want to combine more than two strings you have to use concatenation operator (||).

select 'how' || ' are' || ' you' from dual;
'HOW' || 'ARE

how are you

- **ASCII:** This will return the decimal representation in the database character set of the first character of the string.

Syntax:

ascii
(*string*)

Ex: select ascii('a'),
ascii('apple') from dual;
ASCII('A') ASCII('APPLE')

97

97

- **CHR:** This will return the character having the binary equivalent to

the string in either the database character set or the national character set.

Syntax: chr (*number*)

Ex: select

chr(97) from

dual;

CHR

a

- **SUBSTR:** This will be used to extract substrings. Syntax: substr (*string*, *start_chr_count* [, *no_of_chars*])

Ex: select substr('computer',2), substr('computer',2,5),
substr('computer',3,7) from dual;

SUBSTR(	SUBSTR	SUBSTR
-----	-----	-----
omputer	omput	mputer

Ø If *no_of_chars* parameter is negative then it will display nothing.

Ø If both parameters except *string* are null or zeros then it will display nothing.

Ø If *no_of_chars* parameter is greater than the length of the string then it ignores and calculates based on the original string length.

Ø If *start_chr_count* is negative then it will extract the substring from right end.

1	2	3	4	5	6	7	8
C	O	M	P	U	T	E	R
-8	-7	-6	-5	-4	-3	-2	-1

- **INSTR:** This will allows you for searching through a string for set of characters. Syntax: instr (*string*, *search_str* [, *start_chr_count* [, *occurrence*]])

Ex: select instr('information','o',4,1),

instr('information','o',4,2) from dual;

INSTR('INFORMATION','O',4,1)

INSTR('INFORMATION','O',4,2)

4

10

Ø If you are not specifying *start_chr_count* and *occurrence* then it will start search from the beginning and finds first occurrence only.

Ø If both parameters *start_chr_count* and *occurrence* are null, it will display nothing.

➤ **DATE FUNCTIONS**

Oracle default date format is DD-MON-YY.

We can change the default format to our desired format by using the following command.

```
alter session set nls_date_format = 'DD-  
MONTH-YYYY'; But this will expire once  
the session was closed.
```

- **SYSDATE:** This will give the current date and time. Ex: select sysdate
from dual;
SYSDATE
-

24-DEC-06

- **CURRENT_DATE:** This will returns the current date in the session's timezone. Ex: select current_date from dual;
CURRENT_DATE
-

24-DEC-06

- **CURRENT_TIMESTAMP:** This will return the current timestamp with the active time zone information.

```
Ex: select  
current_timestamp from  
dual;  
CURRENT_TIMESTAMP
```

24-DEC-06 03.42.41.383369 AM +05:30

- **SYSTIMESTAMP:** This will returns the system date, including fractional seconds and time zone of the database.

Ex: select systimestamp
from dual;
SYSTIMESTAMP

24-DEC-06 03.49.31.830099 AM +05:30

- **LOCALTIMESTAMP:** This will returns local timestamp in the active time zone information, with no time zone information shown.

Ex: select
localtimestamp from
dual;
LOCALTIMESTAMP

24-DEC-06 03.44.18.502874 AM

- **DBTIMEZONE:** This will returns the current database time zone in UTC format. (Coordinated Universal Time)

Ex: select
dbtimezone from
dual; DBTIMEZONE

-07:00

- **SESSIONTIMEZONE:** This will returns the value of the current session's time zone. Ex: select sessiontimezone from dual;
SESSIONTIMEZONE
-

+05:30

- **TO_CHAR:** This will be used to extract various date formats. Syntax: to_char (*date*, *format*)

Ex: select to_char(sysdate,'dd month yyyy hh:mi:ss am dy') from dual;
TO_CHAR(SYSDATE,'DD MONTH YYYYHH:MI

24 december 2006 02:03:23 pm sun

```
select to_char(sysdate,'dd month year') from dual;  
TO_CHAR(SYSDATE,'DDMONTHYEAR')
```

```
24 december two thousand six  
select to_char(sysdate,'dd fmmmonth year') from  
dual; TO_CHAR(SYSDATE,'DD FMMONTH  
YEAR')
```

```
24 december two thousand six  
select to_char(sysdate,'ddth DDTH')  
from dual; TO_CHAR(S
```

```
24th 24TH  
select to_char(sysdate,'ddspth DDSPTH') from dual;  
TO_CHAR(SYSDATE,'DDSPTHDDSPTH
```

```
twenty-fourth TWENTY-FOURTH  
select to_char(sysdate,'ddsp Ddsp DDSP ') from dual;  
TO_CHAR(SYSDATE,'DDSPDDSPDDSP')
```

```
twenty-four Twenty-Four TWENTY-FOUR  
• TO_DATE: This will be used to convert the string  
into data format. Ex: select to_char ('24/dec/2006',  
'dd * month * day') from dual;  
TO_CHAR(TO_DATE('24/DEC/20
```

```
24 * december * Sunday  
-- If you are not using to_char oracle will display output in default date  
format.
```

```
• ADD_MONTHS: This will add the specified months  
to the given date. Syntax: add_months (date,  
no_of_months)
```

```
Ex: select add_months ('11-jan-  
1990'), 5) from dual; ADD_MONTHS
```

11-JUN-90
select add_months ('11-jan-1990', -5)
from dual; ADD_MONTH

11-AUG-89
Ø If *no_of_months* is zero then it will display
the same date. Ø If *no_of_months* is null then it
will display nothing.

- **MONTHS_BETWEEN:** This will give difference of months
between two dates. Syntax: months_between (*date1*, *date2*)

Ex: select months_between ('11-aug-1990','11-jan-
1990') from dual; MONTHS_BETWEEN ('11-
AUG-1990','11-JAN-1990')

7
select months_between ('11-jan-1990','11-aug-1990')
from dual; MONTHS_BETWEEN ('11-JAN-1990','11-
AUG-1990')

-7
• **NEXT_DAY:** This will produce next day of the given day from
the specified date. Syntax: next_day (*date*, *day*)
Ex: select next_day ('24-dec-
2006','sun') from dual; NEXT_DAY(

31-DEC-06
-- If the day parameter is null then it will display nothing.
• **LAST_DAY:** This will produce last day of
the given date. Syntax: last_day (*date*)
Ex: select last_day ('24-dec-
2006','sun') from dual; LAST_DAY(

31-DEC-06

- **EXTRACT:** This is used to extract a portion of the date value.

Syntax: extract ((year | month | day | hour | minute | second), *date*) Ex: select extract (year from sysdate) from dual; EXTRACT(YEARFROMSYSDATE)

2006

-- You can extract only one value at a time.